Softwareentwicklung mit Java 2



4. Spezielle Klassen

4. Spezielle Klassen

4.1 Record-Klassen

4.2 Enumerationsklassen

4.3 Innere Klassen

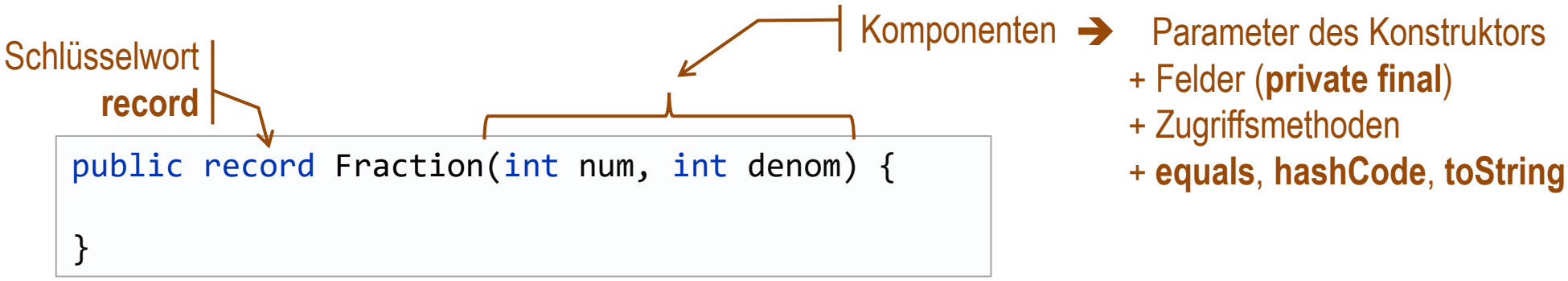
4.4 Fallstudie List

© H. Prähofer, 2026

Record-Klassen

Record-Klassen sind spezielle Klassen für unveränderliche Objekte

- mit Schlüsselwort **record**
- mit **"Komponenten"** = Variablendeklarationen nach Klassennamen



aus Komponenten wird abgeleitet

- **private final** Felder
- **"kanonischen" Konstruktor** mit Paramter gleich den Komponenten
- **Getter-Methoden** für Zugriff auf Felder (mit gleichem Namen wie Komponenten)
- **equals** und **hashCode** auf Basis der Komponenten
- **toString** mit Klassennamen und Werte der Komponenten

© H. Prähofer, 2026

Record-Klassen

```
public record Fraction(int num, int denom) { }
```

Für Record-Klassen ist definiert

- "kanonischer" Konstruktor
  - Konstruktor mit Komponenten als Parameter

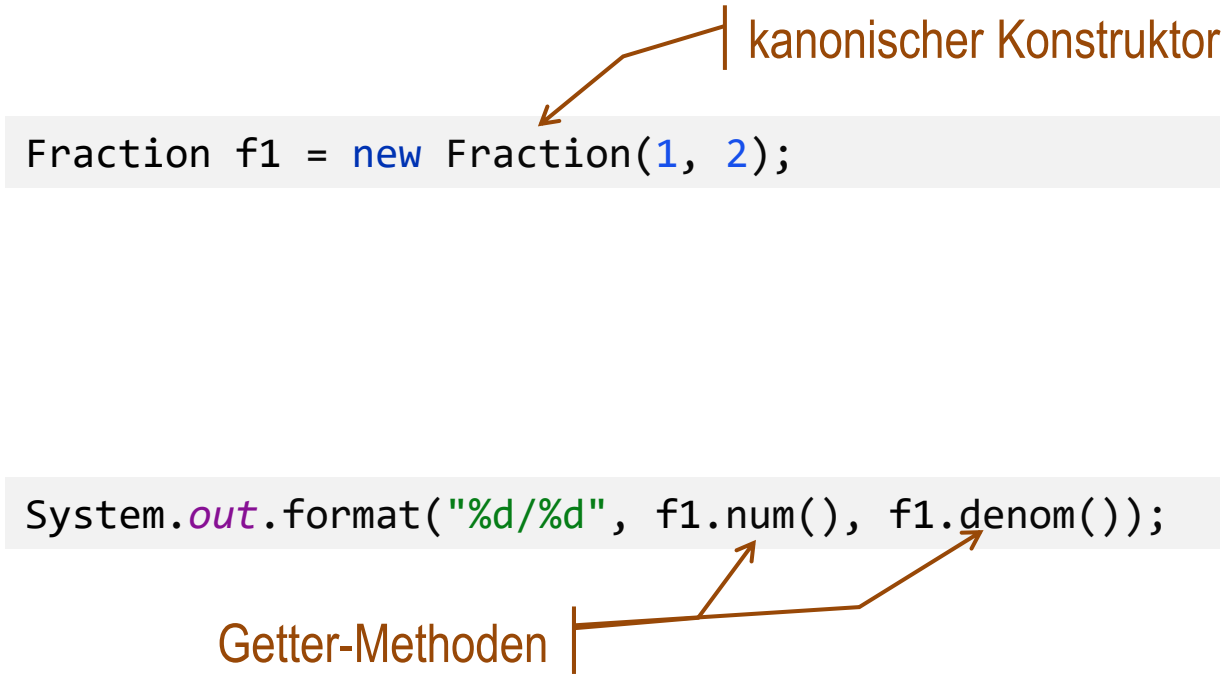
```
public Fraction(int num, int denom) {
  this.num = num; this.denom = denom;
}
```

- Getter-Methoden für Komponenten
  - gleicher Name wie Komponenten

```
public int num() { return num; }
```

```
public int denom() { return denom; }
```

- **equals** und **hashCode**
  - auf Basis der Komponenten
- **toString**
  - auf Basis der Komponenten



© H. Prähofer, 2026

Record-Klassen

Record-Klassen werden expandiert

- Komponenten werden zu **private final** Feldern
- kanonischer Konstruktor, um Felder zu setzen
- Getter-Methoden für Komponenten
- **equals** und **hashCode** mit Vergleich der Felder
- **toString**

```
public record Fraction(int num, int denom) {

}
```

erzeugt

vereinfacht

Record-Klasse ist **final**

```
public final class Fraction extends Record {
   private final int num;
   private final int denom;

   public Fraction(int num, int denom) {
      this.num = num;
      this.denom = denom;
   }

   public int num() {  return num;  }
   public int denom() {  return denom;  }

   @Override
   public boolean equals(Object o) { ... }
   @Override
   public int hashCode() { ... }
      return Objects.hash(num, denom);
   }

   @Override
   public String toString() {
      return getClass() + "[" + num + "," + denom + "]");
   }
}
```

Ableitung von **Record**

Felder **private final**

Konstruktor für Initialisieren der Felder

Zugriffsmethoden für Felder

equals und **hashcode**

toString

© H. Prähofer, 2026

Record-Klassen: Möglichkeiten und Einschränkungen

Bei Record-Klassen ist erlaubt:

+ statische Felder
+ explizite Definition des kanonischen Konstruktor
+ überladene Konstruktoren (müssen kanonischen Konstruktor aufrufen !)
+ "Compact"-Konstruktor
+ Methoden (statische und nicht-statische)
+ Überschreiben der Zugriffsmethoden
+ Implementierung von Interfaces
+ Überschreiben von **toString**
+ Überschreiben von **equals** und **hashCode**

meist nicht notwendig

und nicht erlaubt sind:

– zusätzliche Objektfelder
– Vererbung von Klassen

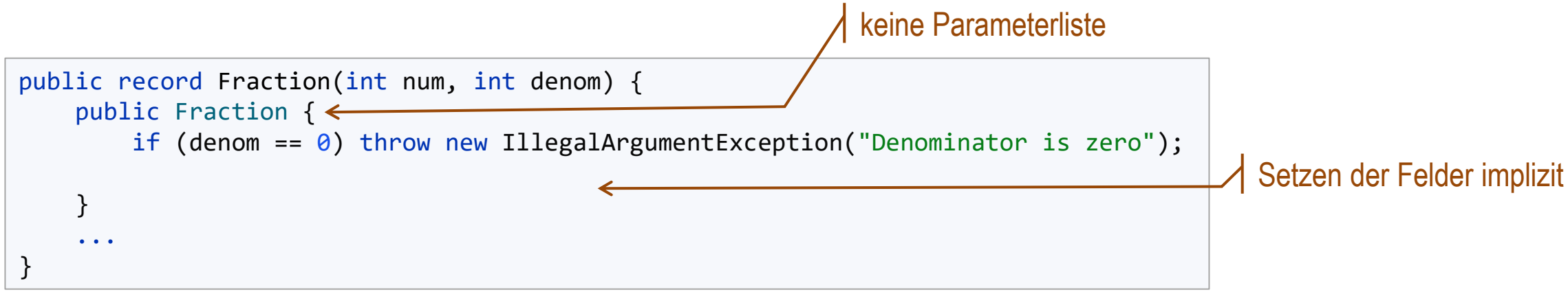
aber Implementierung von **Interfaces** !

© H. Prähofer, 2026

"Compact"-Konstruktor

"Compact"-Konstruktor = Variante zu kanonischem Konstruktor

- keine Parameterliste
- im Rumpf nur Zusätze
  - Setzen der Komponenten wird automatisch angefügt



© H. Prähofer, 2026

Beispiel: Record-Klasse Fraction

```java
public record Fraction(int num, int denom) {

    public Fraction(int num, int denom) {
        if (denom == 0) throw new IllegalArgumentException("Denominator is zero");
        this.num = num;
        this.denom = denom;
    }

    public Fraction(int n) {
        this(n, 1);
    }

    public Fraction mult (Fraction f) {
        return new Fraction(num * f.num, denom * f.denom);
    }
    public Fraction add (Fraction f) {
        return new Fraction(num * f.denom + f.num * denom, n * f.denom);
    }

    @Override
    public String toString() {
        return String.format("%d/%d", num, denom);
    }
}
```

Kanonischer Konstruktor

überladener Konstruktor

Methoden

Überschreiben von toString

© H. Prähofer, 2026

Lernziele überprüfen

- Was sind Record-Klassen in Java und welche besonderen Eigenschaften haben sie?
- Was sind Komponenten bei Record-Klassen und welche Elemente werden daraus automatisch generiert?
- Was versteht man unter dem „kanonischen Konstruktor“ einer Record-Klasse?
- Welche Einschränkungen bestehen für Record-Klassen im Vergleich zu normalen Klassen?

© H. Prähofer, 2026

4. Spezielle Klassen

4.1 Record-Klassen

4.2 Enumerationsklassen

4.3 Innere Klassen

4.4 Fallstudie List

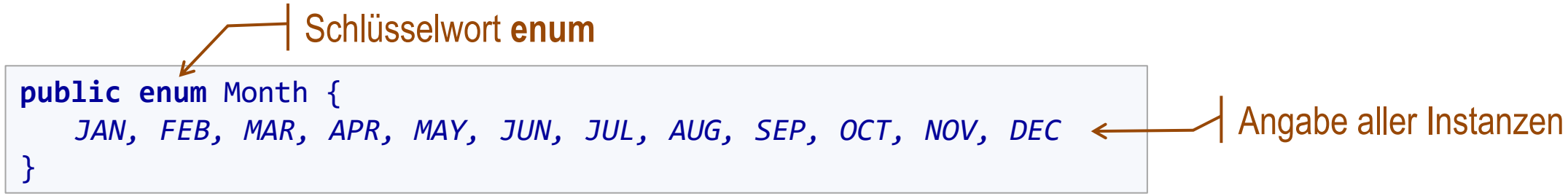
© H. Prähofer, 2026

Enumerationsklassen

Sind spezielle Klassen mit definierten Instanzen

- können immer verwendet werden, wenn **alle Instanzen bekannt** sind

mit einer kompakten syntaktischen Notation



Unterstützte Features

- Zugriff auf Instanzen wie statische Variablen

```
Month month = Month.JUL;
```

- **toString()**

```
Out.println(month.toString());
```

```
JUL
```

- **valueOf**: Enumwert aus String

```
Month august = Month.valueOf("AUG");
```

- **ordinal()**: Position eines Werts

```
Out.println(Month.JAN.ordinal());
Out.println(august.ordinal());
```

```
0
7
```

© H. Prähofer, 2026

Enumerationsklassen

Unterstützte Features (Fortsetzung)

- **values()**: Array mit allen Werten

```
for (Month m : Month.values()) {
  Out.println(m.toString());
}
```

```
JAN
FEB
...
```

- Vergleiche **==** und **!=**

```
if (month == Month.JAN) {
  Out.println(" Exams are coming :-( ");
} else if (month == Month.JUL) {
  Out.println("I am on holiday :-) ");
}
```

```
I am on holiday :-)
```

- Verwendung von **switch**

```
switch (month) {
  case JAN:
    Out.println("Exams are coming :-(");
    break;
  case JUL:
    Out.println("I am on holiday :-)");
    break;
  ...
}
```

```
I am on holiday :-)
```

- implementiert **Comparable** mit Reihenfolge wie Angabe der Werte

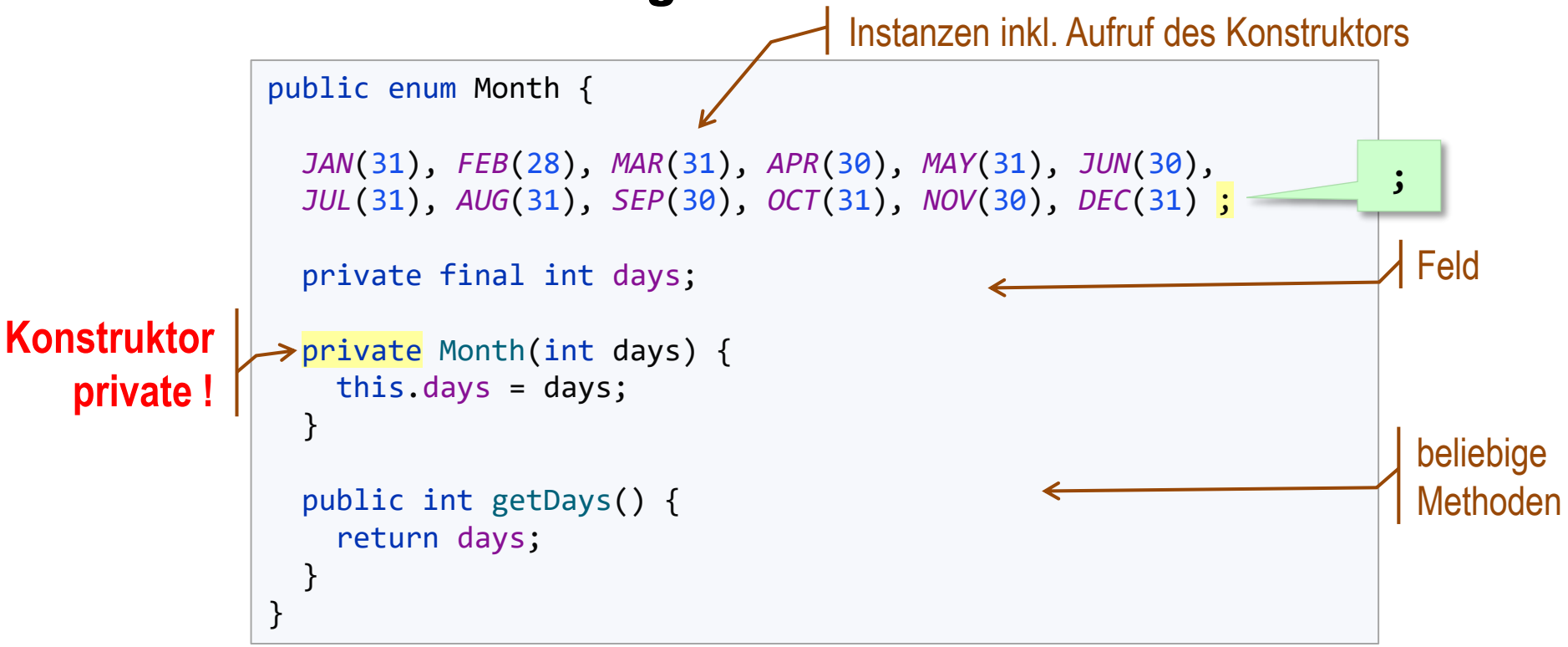
© H. Prähofer, 2026

Enumerationsklassen

Enumerationsklassen können haben

- Felder
- **privaten** Konstruktor
- Methoden

Beispiel: Month mit Anzahl der Tage




```
public enum Month {

  JAN(31), FEB(28), MAR(31), APR(30), MAY(31), JUN(30),
  JUL(31), AUG(31), SEP(30), OCT(31), NOV(30), DEC(31) ;

  private final int days;

  private Month(int days) {
    this.days = days;
  }

  public int getDays() {
    return days;
  }
}
```

© H. Prähofer, 2026

Enumerationsklassen: Möglichkeiten und Einschränkungen

Bei Enumerationsklassen ist erlaubt:

+ Felder deklarieren
+ privaten Konstruktor schreiben
+ Methoden implementieren
+ Methoden überschreiben
+ Implementierung von Interfaces

und nicht erlaubt sind:

– nicht-privaten Konstruktor schreiben
– Objekte selbst instanziieren
– **equals** und **hashCode** überschreiben
– **clone** überschreiben
– **Comparable** implementieren
– von einer Klasse ableiten

aber Implementierung von **Interfaces**

© H. Prähofer, 2026

Lernziele überprüfen

- Was sind Enumerationsklassen in Java und welche besonderen Eigenschaften haben sie?
- Wie werden die Instanzen einer Enumerationsklasse deklariert und verwendet?
- Welche vordefinierten Methoden stellt Enumerationsklassen bereit?
- Welche Möglichkeiten haben Enumerationsklassen hinsichtlich eigener Felder und Methoden?
- Wie muss die Sichtbarkeit des Konstruktors einer Enumerationsklasse sein?
- Welche Einschränkungen gelten für Enumerationsklassen im Vergleich zu normalen Klassen?

© H. Prähofer, 2026

4. Spezielle Klassen

4.1 Record-Klassen

4.2 Enumerationsklassen

4.3 Innere Klassen

- ☐ statische Member-Klassen
- ☐ Member-Klassen
- ☐ lokale Klassen
- ☐ anonyme Klassen

4.4 Fallstudie List

© H. Prähofer, 2026

Top-Level Klassen und Interfaces

- Klassen und Interfaces, die sich direkt in einem Package befinden, sind **Top-Level Klassen** bzw. **Interfaces**
- **Top-Level Klassen** und **Interfaces** können **public** und **package-private** sein

public

```
package p;

public class PublicClass {
  ...
}
```

```
package p;

public interface PublicInterface {
  ...
}
```

package-private

```
package p;

class NonPublicClass {
 ...
}
```

```
package p;

interface NonPublicInterface {
  ...
}
```

© H. Prähofer, 2026

Innere Klassen

Innere Klassen sind lokal zu anderen Klassen

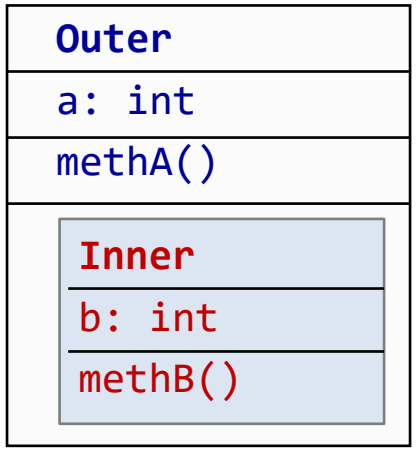
- entweder direkt in Klasse
- oder in Methode einer Klasse

Auch für **Interfaces** mit Einschränkungen

```
public class Outer {
   int a;
   void methA() { ... }

   class Inner {
     int b;
     void methB() { ...}
   }
}
```

UML:



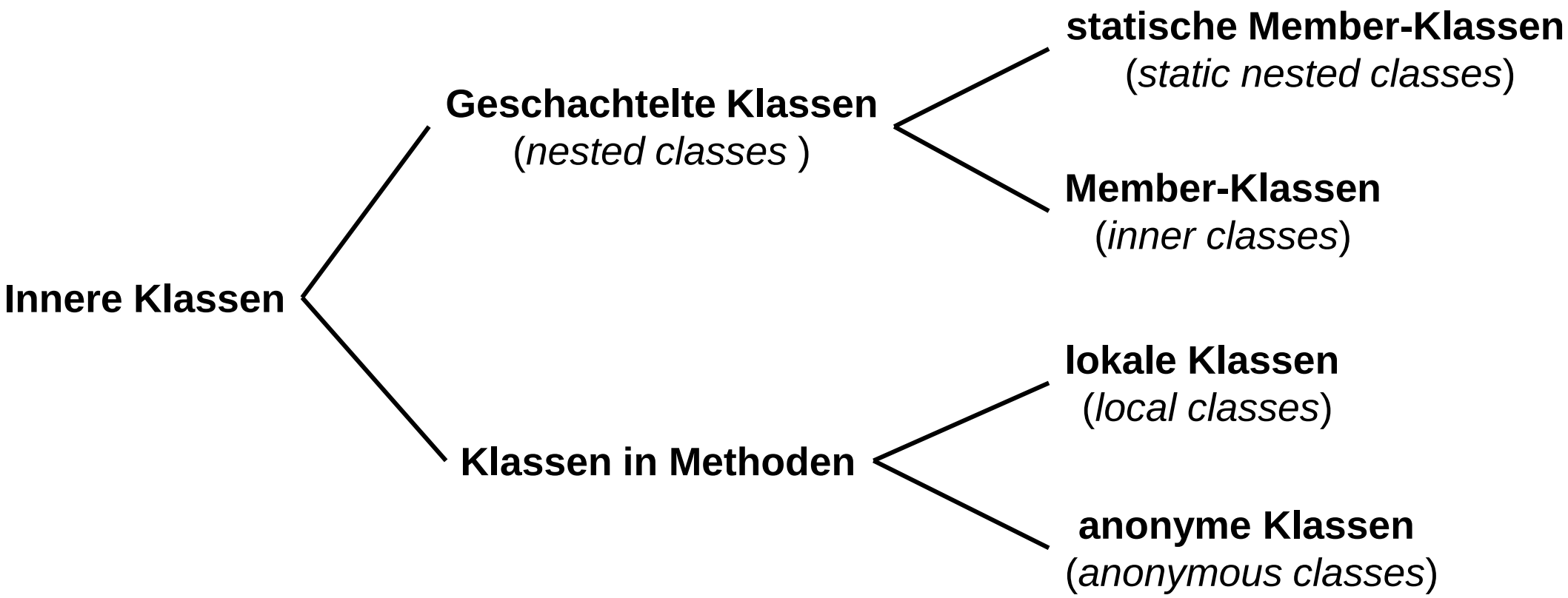
Outer ***äußere*** Klasse
deklarierende Klasse
(***declaring class***)

Inner ***innere*** Klasse
deklarierte Klasse
(***declared class***)

© H. Prähofer, 2026

Klassifikation von inneren Klassen

Für **Interfaces**:
- nur **statische innere Interfaces** möglich
- **Interfaces** können nur **statische innere Klassen** und **innere Interfaces** haben



© H. Prähofer, 2026

Sichtbarkeitsattribute bei geschachtelten Klassen

Für geschachtelte Klassen alle Sichtbarkeitsattribute möglich

- **private**
  - nur innerhalb des Bereichs der äußeren Klasse verwendbar
    - d.h. auch in anderen inneren Klassen!
- **package-private**
  - innerhalb Package sichtbar
- **protected**
  - innerhalb des Bereichs der äußeren Klasse, im gleichen Package und allen Subklassen
- **public**
  - öffentlich zugänglich

Beispiel protected:

Zugriff auf **Node** in Subklasse

```
public class LinkedList {
  protected static class Node {
    ...
  }
  ...
}
```



```
public class ExtLinkedList extends LinkedList {
   public duplicate(Object o) {
     Node dupl = new Node(head.value, head)
     head = dupl;
   }
}
```

© H. Prähofer, 2026

4. Spezielle Klassen

4.1 Record-Klassen

4.2 Enumerationsklassen

4.3 Innere Klassen

- ☐ statische Member-Klassen
- ☐ Member-Klassen
- ☐ lokale Klassen
- ☐ anonyme Klassen

4.4 Fallstudie List

© H. Prähofer, 2026

Statische Member-Klassen

static deklariert

- damit analoge Eigenschaften wie andere **static** Members einer Klasse

```
class OuterClass {
   static class StaticMemberClass { ... }
}
```

selbstständige Klasse innerhalb des „Namensraums“ der deklarierenden Klasse

- ➔ vollqualifizierter Name der Klasse ist ***package.OuterClass.InnerClass***

```
package.OuterClass.StaticMemberClass
```

- haben Zugriff auf alle statischen Members der deklarierenden Klasse (auch private)
- auch statische innere Interfaces möglich

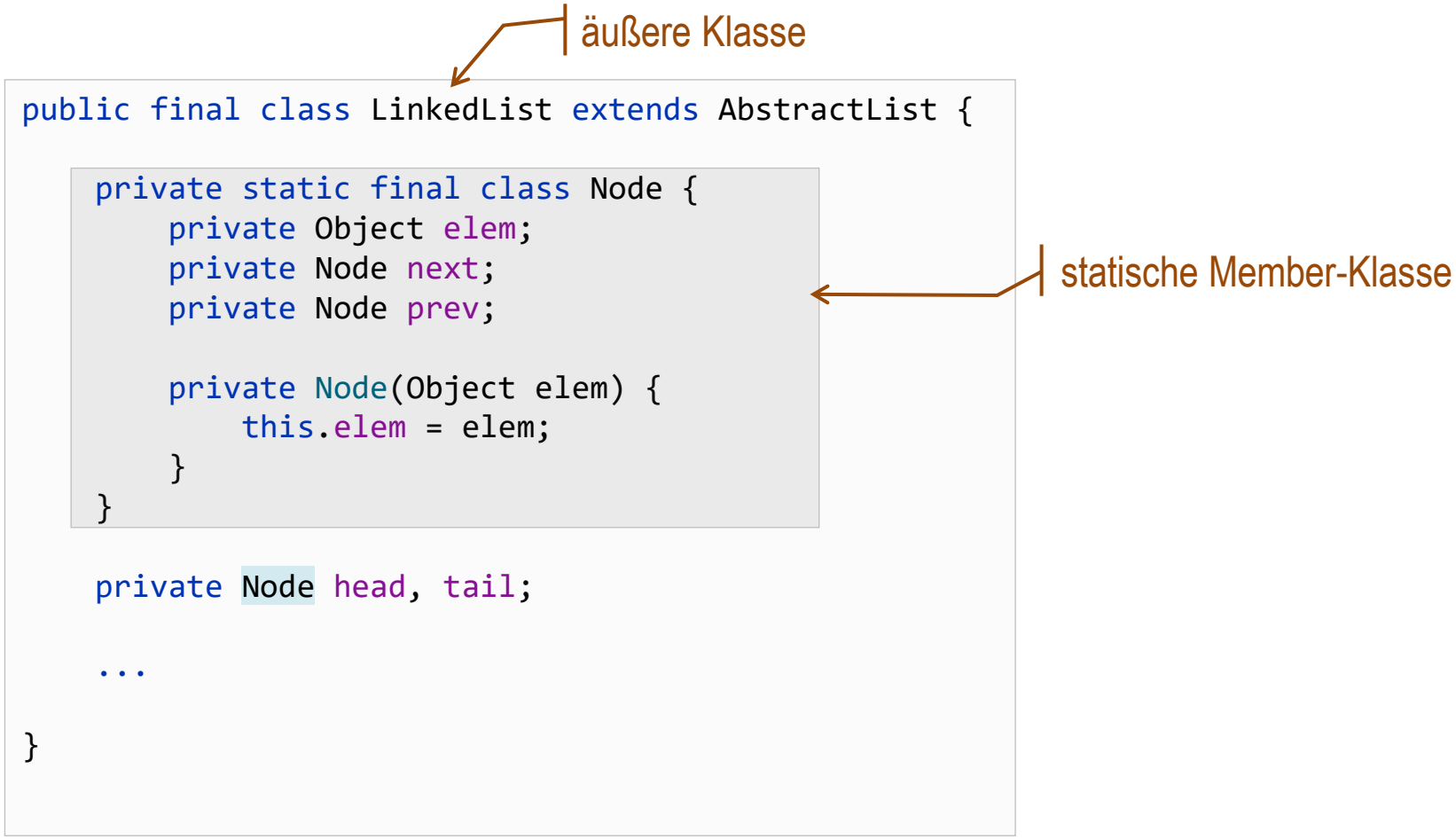
Anwendungen:

- um eine Klasse innerhalb der deklarierenden Klasse zu verbergen
- um eine Klasse oder ein Interface eng an eine äußere Klasse zu koppeln

© H. Prähofer, 2026

Beispiel statische Member-Klasse

Klasse **Node** als statische Member-Klasse, die zu **LinkedList** gehört



© H. Prähofer, 2026

Innere Klassen in Interfaces: Anwendungsbeispiel

Konkrete Klassen die Interfaces implementieren

Beispiel: Shape mit Klassen Rectangle und Circle

```
public interface Shape {
  void draw();

  record Rectangle(int x, int y, int w, int h) implements Shape {
    public void draw() {
      Window.drawRectangle(x, y, w, h);
    }
  }

  record Circle(int x, int y, int r) implements Shape {
    public void draw() {
      Window.drawCircle(x, y, r);
    }
  }
}
```

interface **Shape**

innere Record-Klassen **Rectangle**

und **Circle**

Alle inneren Klassen in Interfaces sind implizit **public static**

© H. Prähofer, 2026

Innere Klassen in Interfaces: Anwendungsbeispiel

- Anwendung

```
public class ShapeMain {
  public static void main(String[] args) {
    Shape.Rectangle r1 = new Shape.Rectangle(10, 10, 100, 200);
    r1.draw();

    Shape.Circle c1 = new Shape.Circle(10, 10, 30);
    c1.draw();
  }
}
```

- mit **import static**

```
import static shapes.Shape.*;

public class ShapeMain {

  public static void main(String[] args) {
    Rectangle r1 = new Rectangle(10, 10, 100, 200);
    r1.draw();

    Circle c1 = new Circle(10, 10, 30);
    c1.draw();
  }
}
```



© H. Prähofer, 2026

4. Spezielle Klassen

4.1 Record-Klassen

4.2 Enumerationsklassen

4.3 Innere Klassen

- ☐ statische Member-Klassen
- ☐ Member-Klassen
- ☐ lokale Klassen
- ☐ anonyme Klassen

4.4 Fallstudie List

© H. Prähofer, 2026

Member-Klassen

Als nicht-statische innere Klassen definiert

```
class OuterClass {

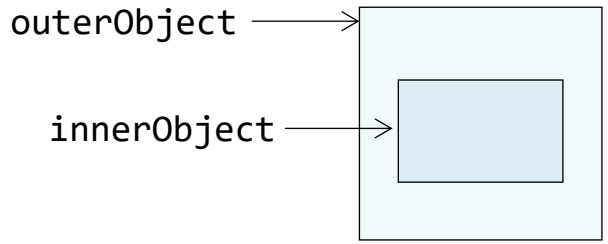
   class InnerClass {
      ...
     }

}
```

- **Objekte der inneren Klasse gehören** immer zu **Objekt der äußeren Klasse**
  ➔ inneres Objekt **"lebt innerhalb"** des äußeren Objekts

- **Innere Objekte** müssen immer mit **äußerem Objekt instanziiert** werden
  ➔ mit speziellem **new**-Operator, der äußeres Objekt definiert

```
OuterClass outerObject = new OuterClass();
OuterClass.InnerClass innerObject = outerObject.new InnerClass(...);
```



© H. Prähofer, 2026

Member-Klassen

- **Innere Objekte** müssen immer mit **äußerem Objekt instanziiert** werden

```
public class Outer {
  int x = 0;
  class Inner {
    int y = 1;
    void incrOuterX() {…}
  }
  public Inner createInner() {
    return this.new Inner();
  }
}
```

inneres Objekt gehört zum äußeren Objekt **this**



```
public Inner createInner() {
  return new Inner();
}
```

this kann ausgelassen werden

© H. Prähofer, 2026

Member-Klassen

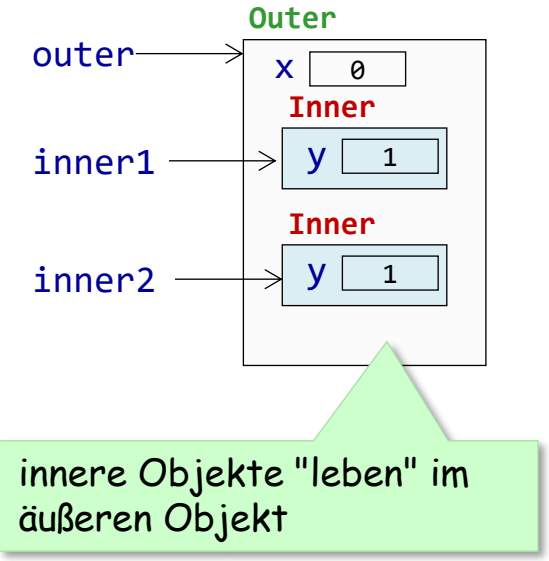
- **Innere Objekte** müssen immer mit **äußerem Objekt instanziiert** werden
- **Innere Objekte "leben"** innerhalb **äußerem Objekt und haben damit Zugriff aus äußeres Objekt**
  ➔ mehrere **innere Objekte** manipulieren **äußeres Objekt**

```
public class Outer {
  int x = 0;
  class Inner {
    int y = 1;
    void incrOuterX() {…}
  }
  public Inner createInner() {
    return this.new Inner();
  }
}
```

inneres Objekt gehört zum äußeren Objekt **this**

```
Outer outer = new Outer();

Outer.Inner inner1 = outer.createInner();
Outer.Inner inner2 = outer.createInner();
...
```



© H. Prähofer, 2026

Member-Klasse

- **Innere Objekte** müssen immer mit **äußerem Objekt instanziiert** werden
- **Innere Objekte "leben"** innerhalb **äußerem Objekt**
- **Innere Objekte** haben **Zugriff** auf **äußeres Objekt**
  - ➔ **Innere Objekte** haben Zugriff auf Objektfelder und -methoden des **äußeren Objekts** (auch auf private)
  - ➔ mehrere **innere Objekte** manipulieren **äußeres Objekt**

```
public class Outer {
  private int x = 0;

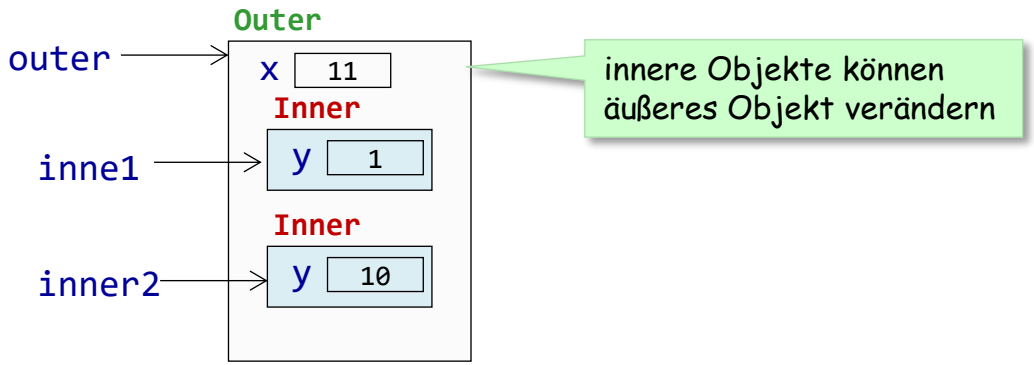
  class Inner {
    int y = 1;

    void incrOuterX() {
      x = x + y;
    }
  }

  public Inner createInner() {
    return new Inner();
  }
}
```

```
Outer outer = new Outer();
Outer.Inner inner1 = outer.createInner();
Outer.Inner inner2 = outer.createInner();

inner2.y = 10;
inner1.incrOuterX();   // outer.x = 1
inner2.incrOuterX();   // outer.x = 11
```



© H. Prähofer, 2026

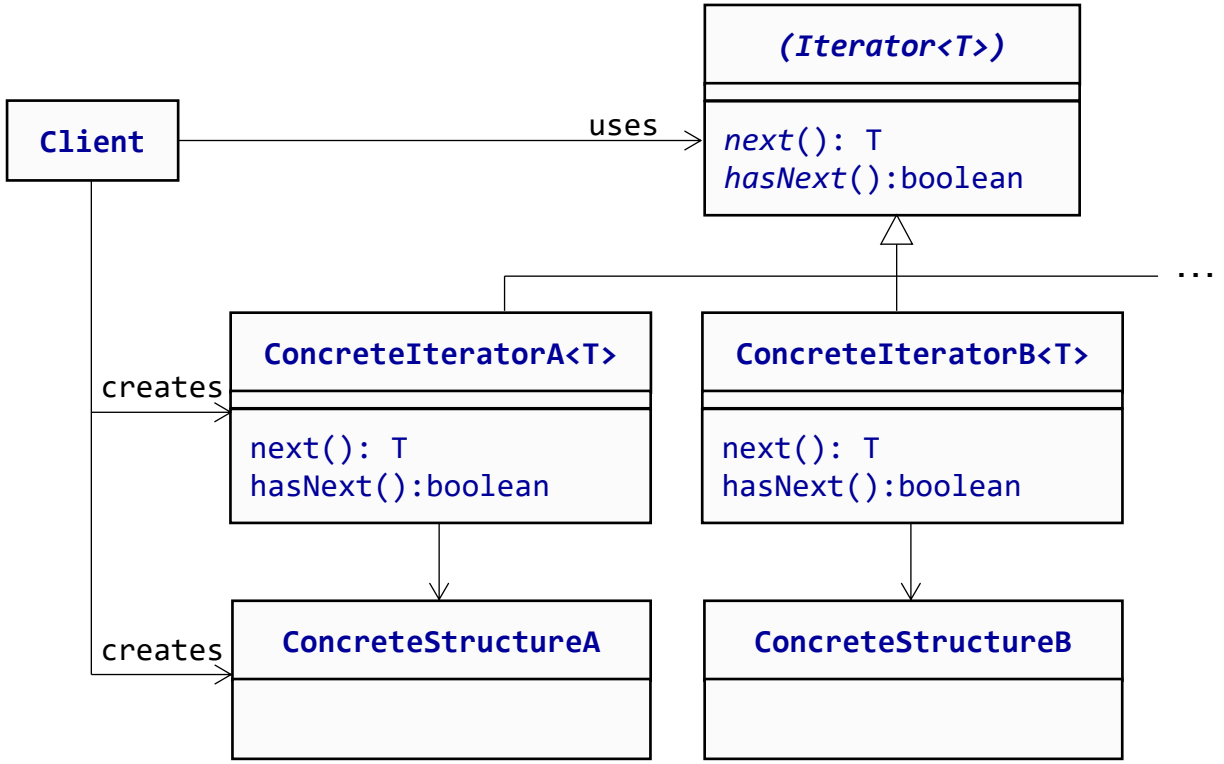
Entwurfsmuster: Iterator

Zweck:

- Auf Elemente einer Struktur sequentiell zugreifen, ohne das Innere der Struktur kennen zu müssen

Lösung:

- Abstrahiere vom konkreten Zugriff und stelle ihn über eine abstrakte Iterator-Schnittstelle bereit mit
  - Zugriff auf das nächste Element
  - Abfrage, ob nächstes Element existiert
- Implementiere Iteratoren für konkrete Strukturen



© H. Prähofer, 2026

Entwurfsmuster Iterator: Interfaces Iterator<T> und Iterable<T>

- **Iterator<T>**: zur Iteration über Collections

```
public interface Iterator<T> {

    boolean hasNext();

    T next();

    default void remove() {
        throw new UnsupportedOperationException("remove");
    }
    ...
}
```

Schleifenkonstruktion mit Iterator:

```
Iterator<Object> it = iterable.iterator();

while (it.hasNext()) {
  Object obj = it.next();
  ... o ...
}
```

- **Iterable<T>**: Interface für alle Typen die Iterator haben

```
public interface Iterable<T> {
    Iterator<T> iterator();
    ...
}
```

forEach-Schleife für alle Iterables:

```
for (Object obj: iterable) {
   ... obj ...
}
```

© H. Prähofer, 2026

Entwurfsmuster Iterator: Schema zur Implementierung

Iteratoren müssen nach folgendem Schema implementiert werden

```
class MyIterator implements Iterator<Object> {

    Feld für Zugriff auf nächstes Element

    MyIterator() {
       Initialisierung des Feldes, sodass erstes Element definiert
    }

    @Override
    public boolean hasNext() {
        return Element ist gültig?
    }

    @Override
    public Object next() {
       if (! hasNext()) throw new NoSuchElementException("no next");
       Object value = Sichern des aktuellen Elements
       Weiterschalten auf nächstes Element
       return value;
    }
}
```

Feld für **Zugriff auf nächstes Element**
➔ nächstes Element muss **bereits vor Zugriff** gesetzt sein

Initialisieren des Feldes, sodass es bereits vor erstem Zugriff auf Element zeigt

Prüfen, ob es nächstes Element gibt

Sichern des aktuell nächsten Elements

Weiterschalten auf nächstes Element

Zurückgeben des gesicherten Elements

© H. Prähofer, 2026

Beispiel Member-Klasse: ListIterator in LinkedList

```
public class LinkedList implements Iterable<Object> {
    private int size;
    private Node head;

    @Override
    public Iterator<Object> iterator() {
        return new ListIterator();
    }

    private final class ListIterator implements Iterator<Object> {

        private Node next = head;

        @Override
        public boolean hasNext() {
           return next != null;
        }

        @Override
        public Object next() {
            if (! hasNext()) throw new NoSuchElementException("no next");
            Object value = next.elem;
            next = next.next;
            return value;
        }
    }
}
```

Interface **Iterable<Object>** implementieren

Methode **iterator** liefert **ListIterator**

Member-Klasse **ListIterator** implemeniert **Iterator<Object>**

Initialisierung von **next** mit erstem Knoten **head**
Zugriff auf **head** des äußeren Objekts

```
LinkedList lst = new LinkedList();
lst.add("A");
...
Iterator<Object> it = lst.iterator();

while (it.hasNext()) {
  Object obj = it.next();
  ... o ...
}
```

© H. Prähofer, 2026

Beispiel Member-Klasse: ListIterator in LinkedList

Schema Implementierung des Iterators: Feld next mit Knoten für nächstes Element

```
public class LinkedList implements Iterable<Object> {

   ...

   private final class ListIterator implements Iterator<Object> {

       private Node next = head;

       @Override
       public boolean hasNext() {
          return next != null;
       }

       @Override
       public Object next() {
           if (! hasNext()) throw new NoSuchElementException("no next");
           Object value = next.elem;
           next = next.next;
           return value;
       }
   }
}
```

Feld **next** mit Knoten für Zugriff auf nächstes Element

Initialisieren des Feldes mit **head**, sodass **next** auf ersten Knoten zeigt

Prüft, ob Knoten gültig

Sichern des Elements

Weiterschalten auf nächsten Knoten

Zurückgeben des gesicherten Elements

© H. Prähofer, 2026

Beispiel Member-Klasse: ListIterator in ArrayList

Index idx für Zugriff auf Array

```
public class ArrayList implements Iterable<Object>  {
  private Object[] elems;
  protected int size = 0;
   ...
  @Override
  public Iterator<Object> iterator() {
      return new ListIterator();
  }

  private final class ListIterator implements Iterator<Object> {

      int idx = 0;

      @Override
      public boolean hasNext() {
        return idx < size;
      }
      @Override
      public Object next() {
        if (! hasNext()) throw new NoSuchElementException("non next");
        Object value = elems[idx];
        idx++;
        return value;
      }
    };
  }
}
```

```
ArrayList lst = new ArrayList();
lst.add("A");
...
Iterator<Object> it = lst.iterator();
while (it.hasNext()) {
  Object obj = it.next();
}
```

Feld **idx** für Index für Zugriff auf nächstes Element

Initialisieren des Feldes **idx** mit 0, sodass es auf erstes Element zeigt

Prüfen, ob es nächsten Index gibt

Sichern des Elements

Weiterschalten auf nächsten Index

Zurückgeben des gesicherten Elements

© H. Prähofer, 2026

4. Spezielle Klassen

4.1 Record-Klassen

4.2 Enumerationsklassen

4.3 Innere Klassen

- ☐ statische Member-Klassen
- ☐ Member-Klassen
- ☐ lokale Klassen
- ☐ anonyme Klassen

4.4 Fallstudie List

© H. Prähofer, 2026

Lokale Klassen

- Lokale Klassen werden **im Block einer Methode** definiert
  - haben damit **lokalen Scope**, d.h. nur innerhalb des Blocks sichtbar und verwendbar
  - Haben Zugriff auf Parameter und lokale Variablen der Methode

Hinweis:
- lokale Klassen werden selten verwendet
- stattdessen meist anonyme Klassen

```
public class SomeClass {

  int x = 0;

  public void foo(int p) {

    class LocalClass {
      ... x ...
      ... p ...
    }
    ...
    LocalClass l = new LocalClass();
    ...
  }

}
```

LocalClass in Methode **foo** definiert

LocalClass hat Zugriff auf Felder der Klasse und Parameter und lokale Variablen der Methode

LocalClass nur innerhalb Methode **foo** sichtbar

© H. Prähofer, 2026

Beispiel lokale Klasse: ListIterator

```
public class LinkedList extends AbstractList implements Iterable<Object> {
    protected Node head, tail;

    @Override
    public Iterator<Object> iterator() {

      class ListIterator implements Iterator<Object> {
        private Node next = head;

        @Override
        public boolean hasNext() { return next != null; }

        @Override
        public Object next() {
            if (! hasNext()) throw new NoSuchElementException("no next");
            Object ret = next.elem;
            next = next.next;
            return ret;
        }
      }

      return new ListIterator();
    }
    ...
}
```

Lokale Klasse **ListIterator** in Methode **iterator**

Klasse **ListIterator** in Methode **iterator** instanziiert

© H. Prähofer, 2026

4. Spezielle Klassen

4.1 Record-Klassen

4.2 Enumerationsklassen

4.3 Innere Klassen

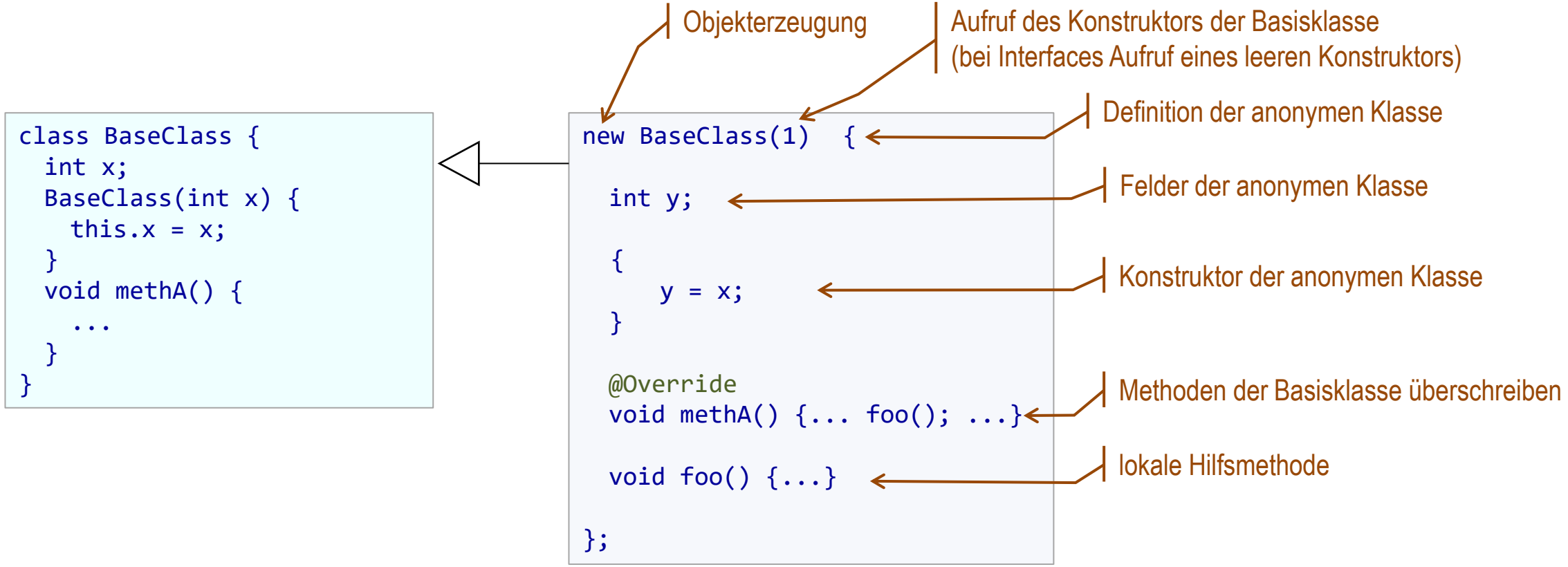
- ☐ statische Member-Klassen
- ☐ Member-Klassen
- ☐ lokale Klassen
- ☐ anonyme Klassen

4.4 Fallstudie List

© H. Prähofer, 2026

Anonyme Klassen

- **Lokale Klasse ohne Namen**
  ➔ werden **direkt bei der Objekterzeugung** definiert
- Anwendung: **Klasse nur bei Objekterzeugung benötigt**
- Definition: Klassencode ausgehend von Basisklasse (auch Interface möglich)



© H. Prähofer, 2026

Beispiel: Anonyme Klasse für Iterator in LinkedList

```
public class LinkedList extends AbstractList implements Iterable<Object> {
    protected Node head, tail;

    @Override
    public Iterator<Object> iterator() {

      return new Iterator<Object>() {
         Node next = head;

         @Override
         public boolean hasNext() { return next != null; }

         @Override
         public Object next() {
           if (! hasNext()) throw new NoSuchElementException("no next");
           Object ret = next.elem;
           next = next.next;
           return ret;
         }

       };

    }
    ...
}
```

Erzeugt anonyme Implementierung von **Iterator<Object>**

© H. Prähofer, 2026

Beispiel: Anonymer Iterator aus Parameter

- Statische Methode **iteratorOf** erzeugt Iterator für Parameter **array** und **size**

```
static Iterator<Object> iteratorOf(Object[] array, int size) {
  return new Iterator<>() {
    private int nextIndex;

    @Override
    public boolean hasNext() {
      return nextIndex < size;
    }

    @Override
    public Object next() {
      if (!hasNext()) throw new NoSuchElementException();
      return array[nextIndex++];
    }
  };
}
```

- **Array-Adapter** verwendet Methode **iteratorOf** mit Element-Array

```
public interface List implements Iterable<Object> {
 static IndexedList of(Object[] array) {
    return new IndexedList() {
      …
      @Override
      public Iterator<Object> iterator() {
        return iteratorOf(elems, elems.length);
      }
    …
```

© H. Pranöter, 2026

this-Zeiger für äußeres Objekt

- In inneren Member-Klassen bezieht sich **this** auf das **innere Objekt**
- Um mit **this** auf das äußere Objekt zuzugreifen, muss man **Klasse des äußeren Objekts** voranstellen

Beispiel:

```
public class LinkedList implements Iterable<Object> {
  private Node head;
  …
  public Iterator<Object> iterator() {

    return new Iterator<Object>() {
      Node next;

      {
        this.next = this.head;
      }

    };
  }
  ...
}
```

this bezieht sich auf Iterator-Objekt
➔ Compile-Fehler

```
public class LinkedList implements Iterable<Object> {
  private Node head;
  …
  public Iterator<Object> iterator() {

    return new Iterator<Object>() {
      Node next;

      {
        this.next = LinkedList.this.head;
      }

    };
  }
  ...
}
```

this zum äußeren Objekt

Gilt für alle inneren Klassen (außer statischen inneren Klassen) !

© H. Prähofer, 2026

Lokale Variablen in anonymen Klassen

- Anonyme Klassen können lokale Variablen nur verwenden, wenn diese **final** oder **effectively final** sind

effectively final =
ohne Modifier **final**
aber niemals verändert!

Beispiel: **Counter**

```
static void main() {

  int count = 0;

  Runnable counter = new Runnable() {
    @Override
    public void run() {
      count = count + 1;
    }
  };

  counter.run();
  counter.run();
  …

  System.out.println(count);
}
```

✗ **Runnable**-Objekt versucht lokale Variable zu setzen ➔ **Compilerfehler**

© H. Prähofer, 2026

Lokale Variablen in anonymen Klassen

- Anonyme Klassen können lokale Variablen nur verwenden, wenn diese **final** oder **effectively final** sind
- Lösung:
  - Heap-Objekt mit veränderlichem Feld
  - Referenz auf Objekt ist **final**
  - Geändert wird das veränderlichen Feldes des Objects

effectively final =
ohne Modifier **final**
aber niemals verändert!

```
static void main() {

  final int[] refCount = new int[1];

  Runnable counter = new Runnable() {
    @Override
    public void run() {
      refCount[0] = refCount[0] + 1;
    }
  };

 counter.run();
 counter.run();
 …
  System.out.println(refCount[0]);
}
```

Referenz auf Array-Objekt ist **final**

Setzen des Feldes des Array-Objekts

© H. Prähofer, 2026

Einschränkungen bei Inneren Klassen und Interfaces

Innere Interfaces

- nur **statische innere Interfaces** möglich

Innere Klassen in Interfaces

- Interfaces können nur **statische innere Klassen** und **Interfaces** haben

Lokale und anonyme Klassen

- können keine **statischen Felder und Methoden** haben

Anonyme Klassen

- zusätzliche Methoden (nicht überschriebene) können nur intern verwendet werden

© H. Prähofer, 2026

Wann verwende ich welche innere Klasse

statische Member-Klasse

- wenn ich eine Hilfsklasse für die innere Implementierung benötige
  ➔ dann **private**, **package-private** oder **protected**
- wenn man eine Klasse eng an eine äußere Klasse oder Interface koppeln will
  ➔ dann auch meist **public**

Member-Klasse

- wenn Objekte der inneren Klasse auf einem Objekt der äußeren Klasse arbeiten sollen
  ➔ wie z.B Iterators auf eine Liste
- und wenn man Objekte an mehreren stellen instanziieren muss
  oder die Klasse sehr umfangreich ist
  ➔ sonst anonyme Klasse

Anonyme Klasse

- wie Member-Klasse aber nur eine Stelle, wo man Objekt erzeugt
  und Klasse nicht zu umfangreich ist

Oft bevorzugt gegenüber Member-Klassen !

© H. Prähofer, 2026

Lernziele überprüfen

- Was versteht man unter einer inneren Klasse?
- Welche Arten von inneren Klassen gibt es in Java?
- Wie ist die Klassifikation von inneren Klassen?
- Welche Sichtbarkeitsattribute sind für geschachtelte Klassen möglich?
- Durch welche Eigenschaften unterscheiden sich nicht-statische Member-Klassen von statischen inneren Member-Klassen?
- Was sind anonyme Klassen und wofür werden sie typischerweise eingesetzt?
- Wie kann man in einer inneren Klasse auf das äußere Objekt zugreifen?
- Warum dürfen in anonymen Klassen nur final- oder effectively-final-Variablen verwendet werden?
- Welche Einschränkungen gelten bei inneren Klassen und Interfaces?
- In welchen Situationen soll man welche innere Klasse verwenden?

© H. Prähofer, 2026

4. Spezielle Klassen

4.1 Record-Klassen

4.2 Enumerationsklassen

4.3 Innere Klassen

4.4 Fallstudie List

© H. Prähofer, 2026

Statische Innere Klasse Node von LinkedList

Klasse Node von LinkedList als statische innere Klasse

- damit nur in LinkedList und Unterklassen sichtbar

```
public final class LinkedList extends AbstractList {

  private static class Node {
    private Object value;
    private Node next, prev;
    private Node(Object value) {
      this.value = value;
    }
  }

  private Node head, tail;

  public LinkedList(Object... initial) {...}

  ...
  @Override
  public void add(Object elem) {
    Node n = new Node(elem);
    link(n, tail, null);
    incrSize();
  }
  ...
}
```

© H. Prähofer, 2026

Implementierung von Iterable und Iterator

- Basisinterface **List** abgeleitet von **Iterable<Object>**
  ➔ alle List-Implementierungen sind damit iterierbar

```
public interface List extends Iterable<Object> {
  ...
}
```

- Methode **iterator** in **LinkedList**

```
public class LinkedList extends AbstractList<T> {
  ...
  @Override
  public Iterator<Object> iterator() {

    return new Iterator<Object>() {
      Node next = head;

      @Override
      public boolean hasNext() {
        return next != null;
      }
      @Override
      public Object next() {
        if (!hasNext()) throw new NoSuchElementException("…");
        Object val = next.value;
        next = next.next;
        return val;
      }
    };
  }
}
```

- Methode **iterator** in **ArrayList<T>**

```
public class ArrayList extends AbstractList {
  ...
  @Override
  public Iterator<Object> iterator() {

    return iteratorOf(elems);
  }
}
```

```
static Iterator<Object> iteratorOf(Object[] array) {
  return new Iterator<>() {
    private int nextIndex;

    @Override
    public boolean hasNext() {
      return nextIndex < array.length;
    }
    @Override
    public Object next() {
      if (!hasNext()) throw new NoSuchElementException();
      return array[nextIndex++];
    }
  };
}
```

© H. Prähofer, 2026

Beispiel: Adapters mit anonymen Klassen

List-Adapters für Array als anonyme Klassen

List-Implementierung als anonyme Klasse

anonyme Iterator-Klasse von anonymer List-Klasse

```
public interface List extends Iterable<Object> {
    ...
    static List of(Object[] array) {
        return new List() {
            @Override public int size() { return array.length; }

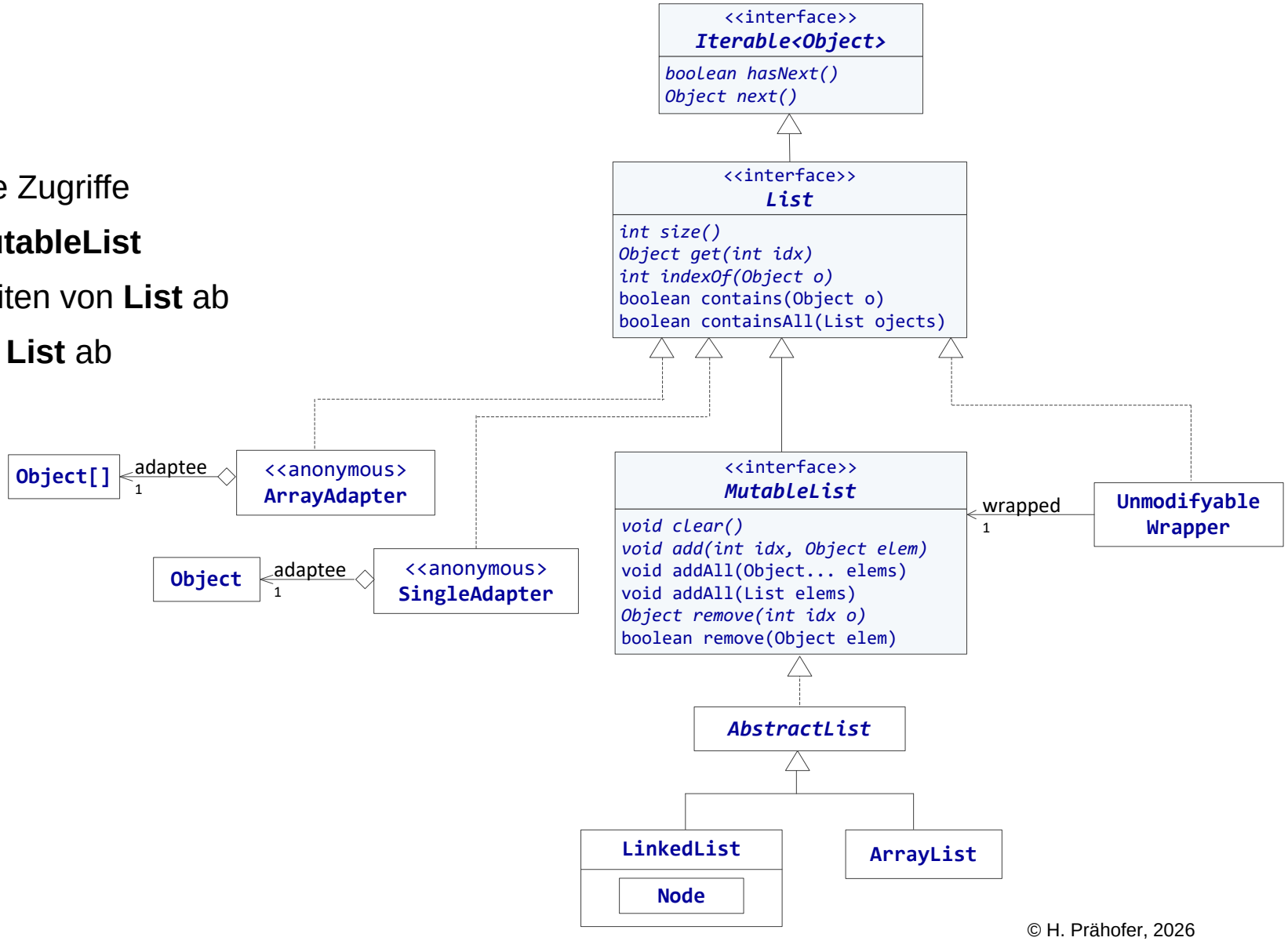
            @Override public int indexOf(Object o) {
                for (int i = 0; i < array.length; i++) {
                    if (array[i].equals(o)) return i;
                }
                return -1;
            }
            ...
            @Override
            public Iterator<Object> iterator() {
                return iteratorOf(array);
            }
        };
    }
}
```

© H. Prähofer, 2026

Interfaces List und MutableList

Interfaces für nur-lesende und verändernde Operationen

- **List** – nur lesende Zugriffe
- **MutableList** – zusätzlich schreibende Zugriffe
- Konkrete Klassen implementieren **MutableList**
- **ArrayAdapter** und **SingleAdapter** leiten von **List** ab
- **UnmodifiableListWrapper** leitet von **List** ab und kapselt **MutableList**



© H. Prähofer, 2026

Problem: Zugriff über Index

- Methode **get** greift über Index auf Element zu
- LinkedList implementiert get mit sequentieller Suche
  - ➔ damit hat Methode lineare Komplexität
  - ➔ und Schleife über Index hat quadratische Komplexität

```
public final class LinkedList extends AbstractList {
  ...
  @Override
  public Object get(int idx) {
    Node n = head;
    for (int i = 0; i < idx; i++) {
      n = n.next;
    }
    return n.elem;
  }
  ...
}
```

Schleife für get !?

```
for (int i = 0; i < l.size(); i++) {
  Object elem = l.get(i);
}
```

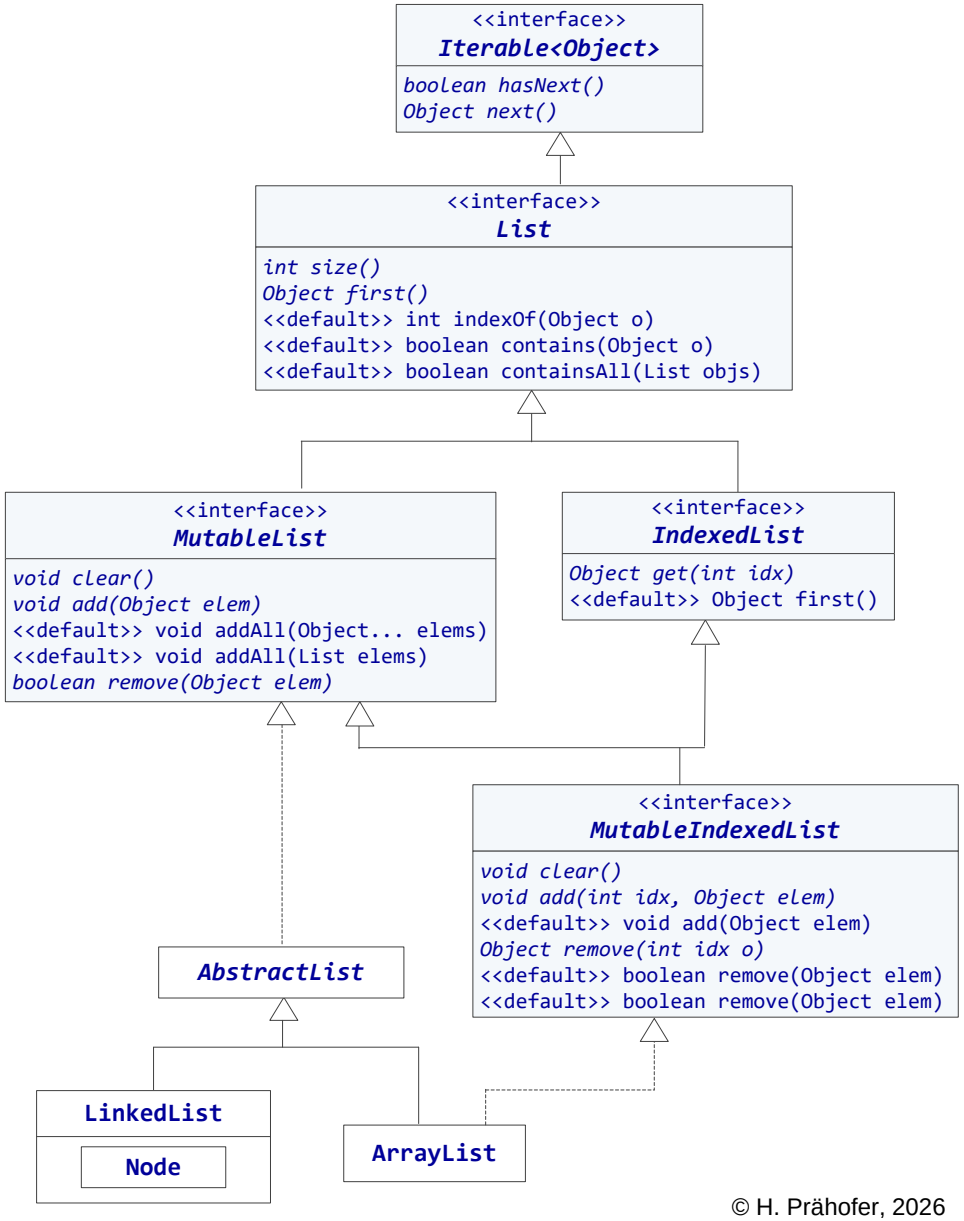
Einfache Schleife mit quadratischer Komplexität!?

➔ **LinkedList** sollte keine Zugriffe über Index haben!

© H. Prähofer, 2026

Interfaces IndexedList und MutableIndexedList

- **List** – sequentieller Zugriff mit Iterator
- **IndexedList** – lesende Zugriffe mit Index
- **MutableList** – Anfügen am Ende
- **MutableIndexedList** – schreibende Zugriff mit Index
- **LinkedList** – implementiert **MutableList**
- **ArrayList** – implementiert **MutableIndexedList**



Beachte:
Große Flexibilität
durch Interfaces!

© H. Prähofer, 2026

Interfaces List und IndexedList

List

```
public interface List extends Iterable<Object> {

  int size();

  Object first();

  default int indexOf(Object o) {
    int i = 0;
    for (Object e: this) {
      if (e.equals(o)) return i;
      i++;
    }
    return -1;
  }

  default boolean contains(Object o) {
    return indexOf(o) >= 0;
  }

  default boolean containsAll(List objs) {
    for (Object o : objs) {
      if (! contains(o)) return false;
    }
    return true;
  }

}
```

IndexedList

```
public interface IndexedList extends List {

Object get(int idx);

  @Override
  default Object first() {
    return get(0);
  }
}
```

© H. Prähofer, 2026

Interfaces MutableList und MutableIndexedList

MutableList

```
public interface MutableList extends List {

  void clear();

  void add(Object elem);

  default void addAll(Object... elems) {
    for (Object e: elems) add(e);
  }


  default void addAll(List elems) {
    for (Object e: elems) add(e);
  }

  boolean remove(Object elem);

}
```

MutableIndexedList

```
public interface MutableIndexedList extends MutableList, IndexedList {

  void add(int idx, Object elem);

  default void add(Object elem) {
    add(size(), elem);
  }


  Object remove(int idx);

  @Override
  default boolean remove(Object elem) {
    int idx = indexOf(elem);
    if (idx >= 0) {
      remove(idx);
      return true;
    } else {
      return false;
    }
  }

}
```

© H. Prähofer, 2026

Lernziele überprüfen

- Wozu dient das Entwurfsmuster Iterator?
- Welche Interfaces gehören zur Iterator-Struktur in Java und welche Methoden stellen sie bereit?
- Wie ist die Schnittstelle der beiden abstrakten Methoden des Interfaces Iterator in Java?
- Skizzieren Sie das grundlegende Schema für die Implementierung eines Iterators in Java.
- Wie funktioniert die for-each-Schleife in Verbindung mit Iterable?
- Nach welchem Schema wird ein Iterator in Java implementiert?
- Wie können anonyme Klassen bei der Implementierung von Adaptern (z. B. List.of(...)) eingesetzt werden?

© H. Prähofer, 2026